

Algorithmique et programmation impérative

Chapitre 6 : Itérations et chaînes de caractères

Hadrien Mélot
Service d'Algorithmique
Département d'Informatique
Faculté des Sciences



Chapitre 6. Itérations et chaînes de caractères

- 1 Incrémenter et décrémenter
- 2 Boucles `while`
- 3 Un `str` est une séquence immuable de caractères
- 4 Boucles `for`
- 5 Invocation de méthodes sur les chaînes
- 6 Opérateurs sur les chaînes
- 7 Utiliser des boucles ou la récursivité ?
- 8 Erreurs fréquentes et debug
- 9 Exercices interactifs
- 10 Pour aller plus loin (à lire par soi-même)
- 11 Glossaire

Et si on commençait le cours par une petite devinette ?

→ **script** `mot_devinette.py`

Incrémenter et décrémenter

Assignment multiple

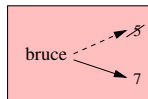
Comme nous l'avons vu, on peut réassigner une valeur à une variable (**assignment multiple**).

```
>>> bruce = 5  
>>> print(bruce)
```

5

```
>>> bruce = 7  
>>> print(bruce)
```

7



Incrémenter et décrémenter

Une des formes les plus courantes de l'assignation multiple : la **mise à jour** d'une variable, où la nouvelle valeur de la variable dépend de son ancienne valeur.

$$x = x + 1$$

“obtenir la valeur courante de x , ajouter 1, et mettre à jour x avec la nouvelle valeur.”

Incrémenter et décrémenter

Si on met à jour une variable qui n'a pas été créée, cela provoque une erreur (Python évalue le côté droit de l'assignation *avant* d'assigner la valeur au membre gauche).

```
>>> x = x + 1
```

```
NameError: name 'x' is not defined
```

Avant de mettre à jour une variable, il faut qu'elle ait été **initialisée**, en général avec une simple assignation.

```
>>> x = 0
```

```
>>> x = x + 1
```

Incrémenter et décrémenter

On dit qu'on **incrémente** une variable quand on met à jour une variable en lui **ajoutant** un nombre (souvent 1).

```
x = x + 1
```

```
step = 2
```

```
y = y + step
```

On dit qu'on **décrémente** une variable quand on met à jour une variable en lui **soustrayant** un nombre (souvent 1).

```
n = n - 1
```


Boucles `while`

Boucles `while`

La récursivité permet d'automatiser des tâches répétitives. Les **boucles** ou **structures itératives** également.

Répéter des tâches identiques ou similaires sans erreur est quelque chose que les ordinateurs font bien, contrairement aux humains.

Une première forme de structure itérative : boucle `while` (“tant que”)

Syntaxe

```
while expression booléenne :  
    instructions
```

Comportement

Les instructions (indentées) sont exécutées de manière répétée tant que l'expression booléenne (la condition) retourne vrai.

Boucles while

Voici une manière **récursive** d'afficher un décompte :

```
>>> def countdown(n):  
    if n == 0:  
        print('Boum')  
    else:  
        print(n, end = ' ')  
        countdown(n - 1)
```

```
>>> countdown(3)  
3 2 1 Boum
```

Remarque : l'utilisation de `end = ' '` comme dernier argument de la fonction `print` permet de remplacer le retour à la ligne (ici, par un espace).

Boucles while

Voici une version **itérative** de la fonction `countdown` :

```
>>> def countdown(n):  
    while n > 0:  
        print(n, end = ' ')  
        n = n - 1  
    print('Boum')
```

```
>>> countdown(3)  
3 2 1 Boum
```

Remarque : tout algorithme récursif peut être réécrit sous une forme itérative, et vice versa (cf. Chap. 5 et preuve de Turing).

Boucles `while`

Plus formellement, voici le flot d'exécution d'une instruction `while` :

- ➊ Evaluer la condition (`True` ou `False`)
- ➋ Si la condition est fausse, sortir de l'instruction `while` et continuer l'exécution à l'instruction suivante
- ➌ Si la condition est vraie, exécuter le **corps** (instructions indentées) de l'instruction `while`, puis retourner à l'étape 1

D'où le nom de **boucle** : dans l'étape 3, on boucle le flot d'exécution vers l'étape 1.

Exemple : Version itérative de la méthode de héron (voir chapitre précédent) :

```
def square_root(a):  
    x = 0.0  
    y = 1.0  
    while not almost_equal(x, y):  
        x = y  
        y = (x + a / x) / 2  
    return x
```

Boucles infinies

Le corps d'une boucle doit mettre à jour les valeurs d'une ou plusieurs variables de telle sorte que la condition soit évaluée à `False` à un moment donné, pour que la boucle se termine.

Dans le cas de `countdown`, on peut prouver facilement que si les préconditions sont respectées (n est un entier positif fini) alors la boucle se termine.

On parle de preuve de l'**arrêt** d'une boucle ou d'un algorithme contenant des boucles. Bien souvent, ces preuves se font par récurrence.

Prouver qu'une boucle s'arrête

```
def countdown(n):  
    while n > 0:           #1  
        print(n, end = ' ') #2  
        n = n - 1          #3  
    print('Boum')
```

Preuve de l'arrêt de `countdown` (par récurrence).

On suppose que n est un entier ≥ 0 (fini). Si $n = 0$, la condition (1) est évaluée à faux, et la boucle se termine.

Supposons que la boucle s'arrête pour $n \leq k$ ($k \geq 0$), et prouvons le pour $n = k + 1$. Quand $n = k + 1$, l'instruction 2 est exécutée (affiche $k + 1$), puis $n = k$ lors de l'instruction 3.

La boucle est répétée avec $n = k$. Celle-ci s'arrêtera par hypothèse de récurrence. □

Prouver qu'une boucle s'arrête – un cas difficile

Prouver qu'une boucle s'arrête n'est pas toujours facile. Le cas suivant est un exemple (heureusement très rare).

```
def sequence(n):  
    while n != 1:  
        print(n, end = ' ')  
        if n % 2 == 0:  
            n = n // 2  
        else:  
            n = n * 3 + 1
```

- n diminue quand il est pair, mais augmente quand il est impair : une preuve simple par récurrence ne marcherait pas ;
- pour certaines valeurs de n , il est facile de prouver qu'on atteint 1 (par ex., si n est une puissance de 2) ;
- mais prouver le cas général (tout entier fini strictement positif) est-il facile ?

Prouver qu'une boucle s'arrête – un cas difficile

Définition

Pour un nombre naturel strictement positif n donné, on peut appliquer l'opération suivante :

- si n est pair, on le divise par 2 ;
- si n est impair, on le multiplie par 3 et on ajoute 1.

La **suite de Syracuse** de n est la suite de nombres naturels obtenue en répétant successivement l'opération ci-dessus.

Exemple : à partir de 7, on obtient la suite

7, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1, 4, 2, 1, 4, 2, 1, ...

Prouver qu'une boucle s'arrête – un cas difficile

Conjecture (Conjecture de Syracuse ou de Collatz)

La suite de Syracuse d'un nombre naturel strictement positif quelconque n atteint toujours 1.

Cette conjecture constitue un problème ouvert depuis 1952 !

« Les mathématiques ne sont pas encore prêtes pour de tels problèmes » (Erdős)

Une **conjecture** est un énoncé qui semble vrai, mais pour lequel on n'a pas encore pu

- prouver mathématiquement sa validité ;
- trouver de contre-exemple montrant qu'il est faux.

Un `str` est une séquence
immuable de caractères

Une chaîne de caractères est une séquence

Un `str` est une **séquence** de caractères (collection ordonnée).

On peut accéder à chaque caractère en utilisant l'opérateur `[.]` (crochets droits, "bracket").

L'opérateur `[x]` — où `x` est un entier — retourne un `str` constitué d'un seul caractère : celui se trouvant à l'indice `x`

Exemple :

```
>>> fruit = 'banane'
>>> lettre = fruit[1]
>>> type(lettre)
<class 'str'>
```

Question : que va afficher la ligne suivante ?

```
>>> print(lettre)
```

Indices d'une chaîne de caractères

```
>>> fruit = 'banane'
>>> lettre = fruit[1]
>>> print(lettre)
```

a

La première lettre du mot banane est “b”, pas “a”. Mais, en informatique, les indices d'une séquence commencent souvent à 0. C'est le cas également en Python.

```
>>> lettre = fruit[0]
>>> print(lettre)
```

b

Indices d'une chaîne de caractères

```
>>> fruit = 'banane'
>>> lettre = fruit[1]
>>> print(lettre)
```

a

La première lettre du mot banane est “b”, pas “a”. Mais, en informatique, les indices d'une séquence commencent souvent à 0. C'est le cas également en Python.

```
>>> lettre = fruit[0]
>>> print(lettre)
```

b

Intuition : indice = nombre de caractères qui séparent le caractère considéré du début de la séquence

b	a	n	a	n	e
0	1	2	3	4	5

Indices d'une chaîne de caractères

La **longueur** d'une séquence est son nombre d'éléments. Elle peut être obtenue grâce à la fonction `len` ("length").

Indices d'une chaîne de longueur n : entier allant de 0 à $n - 1$.

```
>>> mot = 'computer'
>>> len(mot)
8
>>> mot[0]
'c'
>>> mot[7]
'r'
>>> mot[8]
IndexError: string index out of range
>>> mot[1.5]
TypeError: string indices must be integers, not float
```


Indices d'une chaîne de caractères

On peut également accéder aux éléments grâce à un **indice inversé** : le dernier caractère s'obtient avec l'indice -1 , l'avant dernier avec -2 , etc.

Indices inversés d'une chaîne de longueur n : entier allant de -1 à $-n$.

```
>>> mot = 'banane'
>>> mot[-1]
'e'
>>> mot[-6]
'b'
>>> mot[-7]
IndexError: string index out of range
```

0	1	2	3	4	5
b	a	n	a	n	e
-6	-5	-4	-3	-2	-1

Indices d'une chaîne de caractères

Accéder au dernier élément d'une chaîne est pratique via l'indice -1 , mais pour les autres éléments, l'utilisation est plutôt rare et nous l'éviterons dans ce cours.

```
>>> mot = 'banane'
>>> mot[-1]
'e'
```

Exercices

Problème

Ecrire une fonction qui prend un mot en paramètre et retourne une chaîne qui a remplacé les 'a' par 'o' dans le mot.

Exemple :

`a2o('haha')` $\longrightarrow$ `'hoho'`

Exercices

Ramplacer les 'a' par 'o' :

```
def a2o(s):  
    res = '' # chaine vide  
    i = 0  
    while i < len(s):  
        if s[i] == 'a':  
            res = res + 'o'  
        else:  
            res = res + s[i]  
        i = i + 1  
    return res
```

Exercices

Problème

Ecrire une fonction qui prend un mot et une lettre en paramètres et retourne l'indice de la première occurrence de la lettre dans le mot, ou `None` si la lettre n'est pas présente.

Exemples :

```
find('banane', 'a') → 1
```

```
find('banane', 'm') → None
```

Exercices

Trouver l'indice de la première occurrence d'un caractère :

```
def find(word, letter):  
    i = 0  
    while i < len(word):  
        if word[i] == letter:  
            return i  
        i = i + 1  
    return None
```

Rappel : `return` provoque une sortie directe de la fonction. C'est également le cas dans une boucle. Donc ici, dès que la lettre est trouvée, on sort de la fonction en retournant l'indice.

Exercices

La fonction `find` fait donc le contraire de l'opérateur `[.]`, sauf qu'elle ne retourne que le **premier** indice d'une lettre donnée.

```
>>> mot = 'banane'
>>> find(mot, 'a')
1
>>> mot[1]
'a'
>>> mot[3]
'a'
```

Exercices

Problème

Ecrire une fonction qui prend une chaîne en argument et qui retourne une chaîne constituée des caractères de la chaîne dont l'ordre est inversé.

Exemple : `invert('algorithm')` $\longrightarrow$ `'emhtirogla'`

Exercices

Inversion d'une chaîne :

```
def invert(s):  
    res = ''  
    i = len(s) - 1  
    while i >= 0:  
        res = res + s[i]  
        i = i - 1  
    return res
```

Exercices

Problème

Ecrire une fonction qui prend un nombre binaire (entier) en argument (représenté sous la forme d'une chaîne de caractères) et qui retourne ce nombre en base 10 (sous la forme d'un entier). Cette fonction retourne `None` si la chaîne ne représente pas un nombre binaire.

Exemples :

`bin2dec('10011')` $\longrightarrow$ 19 `bin2dec('123')` $\longrightarrow$ None

En effet,

$$1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 16 + 2 + 1 = 19$$

Exercices

Si `b = '10011'`, alors `bin2dec(b)` $\rightarrow$ 19 car

$$1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 16 + 2 + 1 = 19$$

Soit `i` un indice pour traverser `b` et `p` un entier représentant la puissance à un indice donné :

<code>i</code>	0	1	2	3	4
<code>b[i]</code>	1	0	0	1	1
<code>p</code>	4	3	2	1	0

Pour un `i` donné, que vaut `p` ?

Exercices

Si `b = '10011'`, alors `bin2dec(b)` $\rightarrow$ 19 car

$$1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 16 + 2 + 1 = 19$$

Soit `i` un indice pour traverser `b` et `p` un entier représentant la puissance à un indice donné :

<code>i</code>	0	1	2	3	4
<code>b[i]</code>	1	0	0	1	1
<code>p</code>	4	3	2	1	0

Pour un `i` donné, que vaut `p`?

$$p = \text{len}(b) - i - 1$$

Exercices

Conversion binaire → décimal

```
def bin2dec(b):  
    res = 0  
    i = 0  
    while i < len(b):  
        if b[i] == '1':  
            res = res + 2 ** (len(b)-i-1)  
        elif b[i] != '0':  
            return None  
        i = i + 1  
    return res
```

Tranches de chaînes

Un segment d'une chaîne est appelé une **tranche** ("slice"). On peut accéder à une tranche d'une chaîne avec l'opérateur `[n:m]` où

- `n` est l'indice du premier caractère de la tranche (inclus dans celle-ci)
- `m` est l'indice du dernier caractère de la tranche (**non inclus** dans celle-ci)

```
>>> mot = 'computer'
>>> mot[1:4]
'omp'
>>> mot[0:len(mot)]
'computer'
```

Intuition : on peut voir l'opérateur `[n:m]` comme l'intervalle $[n, m[$ (fermé – ouvert)

Remarque : le nombre de caractères de la tranche est égal à $m - n$ (quand les indices sont positifs)

Tranches de chaînes

Si le premier indice est omis, la tranche commence au début de la chaîne (équivalent à 0)

Si le deuxième indice est omis, la tranche se termine à la fin de la chaîne (équivalent à `len(.)`)

```
>>> mot = 'computer'
>>> mot[:4]
'comp'
>>> mot[4:]
'uter'
>>> mot[:-1]
'compute'
>>> mot[:]
'computer'
```

Tranches de chaînes

Si le premier indice est plus grand ou égal au deuxième indice, le résultat est une **chaîne vide**, représentée par deux apostrophes.

```
>>> mot = 'computer'
>>> mot[3:3]
''
```

Une chaîne vide ne contient pas de caractère et a une longueur 0. A part ça, c'est une chaîne de caractères comme les autres.

Exercices

Problème

Améliorez la fonction `bin2dec` pour gérer également les nombres binaires fractionnaires.

Exemples :

`bin2dec('11')` $\rightarrow$ 3

`bin2dec('.01')` $\rightarrow$ 0.25

En effet,

$$0 \times \frac{1}{2^1} + 1 \times \frac{1}{2^2} = \frac{1}{4} = 0.25$$

`bin2dec('10011.01')` $\rightarrow$ 19.25

`bin2dec('3.1416')` $\rightarrow$ **None**

Exercices

`bin2dec('10011.01')` $\longrightarrow$ 19.25

On va utiliser notre fonction `find` pour chercher la présence d'un point et séparer la chaîne en une tranche représentant la partie entière et une tranche celle fractionnaire.

On va ensuite traiter, via deux fonctions distinctes, les parties entières et fractionnaires.

`bin2dec_intPart('10011')` $\longrightarrow$ 19

`bin2dec_fracPart('01')` $\longrightarrow$ 0.25

Exercices

L'ancien code de `bin2dec` est placé dans une fonction annexe.

```
def bin2dec_intPart(b):  
    res = 0  
    i = 0  
    while i < len(b):  
        if b[i] == '1':  
            res = res + 2 ** (len(b)-i-1)  
        elif b[i] != '0':  
            return None  
        i = i + 1  
    return res
```

Exemple :

`bin2dec_intPart('10011')` $\longrightarrow$ 19

Exercices

On crée une deuxième fonction annexe pour gérer la partie fractionnaire

```
def bin2dec_fracPart(b):  
    f = 0.0  
    i = 0  
    while i < len(b):  
        if b[i] == '1':  
            f = f + 1.0 / (2 ** (i+1))  
        elif b[i] != '0':  
            return None  
        i = i + 1  
    return f
```

Exemple :

`bin2dec_fracPart('01')` $\longrightarrow$ 0.25

Exercices

On gère le cas général.

```
def bin2dec(b):
    indexDot = find(b, '.')
    if indexDot == None:
        return bin2dec_intPart(b)
    i = bin2dec_intPart(b[:indexDot])
    f = bin2dec_fracPart(b[indexDot + 1:])
    if i != None and f != None:
        return i + f
    else:
        return None
```

Exemples :

`bin2dec('101')` $\longrightarrow$ 5

`bin2dec('.01')` $\longrightarrow$ 0.25

`bin2dec('10011.01')` $\longrightarrow$ 19.25

`bin2dec('3.101')` $\longrightarrow$ **None**

Une chaîne est immuable

On peut être tenté d'utiliser l'opérateur `[.]` dans le membre gauche d'une assignation, pour modifier un de ses caractères.

```
>>> greeting = 'bonjour'
>>> greeting[0] = 'B'
TypeError: 'str' object does not support item assignment
```

Cela provoque une erreur : les chaînes sont **immuables**, c-à-d qu'on ne peut pas modifier une chaîne existante.

Solution : créer une nouvelle chaîne et assigner celle-ci à l'ancienne variable.

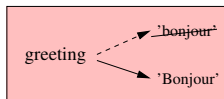
```
>>> greeting = 'bonjour'
>>> new_greeting = 'B' + greeting[1:]
>>> greeting = new_greeting
>>> print(greeting)
Bonjour
```

Une chaîne est immuable

Remarques :

- on peut se passer de la variable `new_greeting`
- l'assignation ne modifie pas la chaîne de départ (valeur) mais précise qu'une nouvelle valeur doit être référencée (le membre droit d'une assignation étant évalué avant d'affecter la valeur obtenue)

```
>>> greeting = 'bonjour'
>>> greeting = 'B' + greeting[1:]
>>> print(greeting)
Bonjour
```



Boucles for

Objets itérables

Une chaîne de caractère est un objet **itérable**, c'est à dire qu'on peut facilement accéder à chaque élément de la séquence, un par un.

Le moyen le plus simple d'itérer est d'utiliser une boucle `for` : chaque **itération** de la boucle accédera à un élément de la séquence via une variable qui est mise à jour à chaque itération.

```
>>> for lettre in 'bonjour':  
    print(lettre, end = ' ', ' ')
```

```
b, o, n, j, o, u, r,
```

Boucles for

Syntaxe

```
for variable in itérable :  
    instructions
```

Comportement

Le corps de la boucle (instructions indentées) est exécuté de manière répétée pour chaque élément de l'objet itérable : à chaque itération, un élément est affecté à la variable.

Exemple

```
>>> s = 'hello'  
>>> res = ''  
>>> for c in s:  
    res = res + c + c  
  
>>> print(res)  
hheelllloo
```

Boucles for

Il existe beaucoup de types d'objets itérables en Python.
Par exemple : les tuples, les listes (ch. 7) et les fichiers (ch. 10).

```
>>> t = (2, 3)
>>> type(t)
<class 'tuple'>
>>> res = 0
>>> for x in t:
    res = res + x

>>> print(res)
5
```

Objets range

La fonction `range(n, m)` retourne un objet `range` qui, quand il est itéré, permet d'accéder aux entiers de l'intervalle `[n, m[`.

Les objets `range` sont également itérables.

```
>>> type(range(1, 4))  
<class 'range'>  
>>> for i in range(1, 4):  
    print(i, end = ' ')
```

```
1 2 3
```

Remarque : en Python 2, la fonction `range` retourne une liste au lieu d'un objet `range`.

Objets range

Si un seul paramètre `m` est donné à la fonction `range`, l'intervalle est `[0, m[`.

```
>>> for i in range(5):  
    print(i, end = ' ')
```

```
0 1 2 3 4
```

```
>>> s = 0
```

```
>>> for i in range(6):  
    s = s + i
```

```
>>> print(s)
```

```
15
```

```
>>> for i in range(3):  
    print('he', end = '')
```

```
hehehe
```

Exercice

Problème

Ecrire une fonction qui compte le nombre d'occurrences d'une lettre donnée dans une chaîne.

Exemple :

`nbr_occurences('banane', 'n') → 2`

Exercice

```
def nbr_occurences(word, letter):  
    cnt = 0  
    for char in word:  
        if char == letter:  
            cnt = cnt + 1  
    return cnt
```

Invocation de méthodes sur les chaînes

Objets

Dans ce cours, on a déjà évoqué la notion d'objet (par ex., objets “module” et objets “fonctions”).

```
>>> import math
>>> print(math)
<module 'math' from '/Library/python2.6/lib-dynload/math.so'>
>>> print(nbr_occurences)
<function nbr_occurences at 0xead7b0>
```

Objets

C'est limitatif mais — pour le moment — considérons un objet comme un valeur (assignée à une variable) possédant des **attributs**.

Exemple : Un objet de type “voiture” pourrait avoir les attributs : marque, cylindrée, prix, couleur, etc.

La notation point permet d'accéder aux attributs d'un objet

Exemple : variables et fonctions d'un objet module.

```
>>> math.sin(math.pi / 2.0)  
1.0
```

Objets

La fonction `dir` permet d'afficher la liste des attributs d'un objet.

```
>>> dir(nbr_occurences)
['__annotations__', '__call__', '__class__', '__closure__', '__code__',
 '__defaults__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__',
 '__format__', '__ge__', '__get__', '__getattr__', '__globals__',
 '__gt__', '__hash__', '__init__', '__kwdefaults__', '__le__', '__lt__',
 '__module__', '__name__', '__ne__', '__new__', '__qualname__',
 '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__',
 '__str__', '__subclasshook__']
>>> print(nbr_occurences.__name__)
nbr_occurences
```

Objets

En réalité, **en Python, tout est objet**. Cela implique que tout peut être assigné à une variable ou passé comme argument à une fonction, même une fonction ou un module.

Les entiers et les chaînes de caractères sont donc aussi des objets.

```
>>> s = 'bonjour'
>>> dir(s)
['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__',
...
'capitalize', 'casefold', 'center', 'count', 'encode', 'endswith', 'expandtabs',
'find', 'format', 'format_map', 'index', 'isalnum', 'isalpha', 'isdecimal',
'isdigit', 'isidentifier', 'islower', 'isnumeric', 'isprintable', 'isspace',
'istitle', 'isupper', 'join', 'ljust', 'lower', 'lstrip', 'maketrans',
'partition', 'replace', 'rfind', 'rindex', 'rjust', 'rpartition', 'rsplit',
rstrip', 'split', 'splitlines', 'startswith', 'strip', 'swapcase', 'title',
'translate', 'upper', 'zfill']
```

Objets

Les attributs sont liés au type des objets, c'est à dire qu'ils sont

- identiques pour tous les objets d'un même type ;
- spécifiques au type de l'objet, donc deux objets de deux types différents n'ont pas la même liste d'attributs.

En séance : illustration des sorties des appels à `dir` suivants :

```
>>> s = 'bonjour'
>>> dir(s)
...
>>> dir(str)
...
>>> dir('')
...
>>> dir(int)
...
>>> dir(12)
...
```

Méthodes

Les attributs d'un objet `str` sont nombreux et de types divers, mais la plupart d'entre eux sont des **méthodes**.

```
>>> s = 'bonjour'
>>> type(s.__doc__)
<class 'str'>
>>> type(s.upper)
<class 'builtin_function_or_method'>
>>> type('hello'.find)
<class 'builtin_function_or_method'>
```

Une **méthode** est similaire à une fonction — elle prend des arguments et retourne une valeur — mais elle s'applique **sur** un objet, et la syntaxe est donc légèrement différente.

Invocations de méthodes

Pour obtenir une (nouvelle) chaîne en majuscules à partir d'une chaîne, on peut :

- écrire, puis utiliser une fonction

```
>>> def set_upper(s):  
    ...  
  
>>> res = set_upper('bonjour')  
>>> print(res)  
'BONJOUR'
```

- invoquer la méthode `upper` qui est disponible pour tous les objets de type `str`

```
>>> s = 'bonjour'  
>>> res = s.upper()  
>>> print(res)  
'BONJOUR'  
>>> print('hello'.upper())  
'HELLO'
```

Différence de syntaxe entre fonctions et méthodes

On parle d'**appel** à une fonction. Une fonction a besoin de toutes ses entrées en paramètres (ici, une chaîne).

```
>>> res = set_upper('bonjour')
```

On dit qu'on **invoque** une méthode **sur** un objet.

```
>>> res = 'bonjour'.upper()
```

La notation point permet de préciser sur quel objet la méthode est invoquée. Il n'est donc pas nécessaire de passer cet objet en paramètre.

Excepté cette différence de syntaxe, les méthodes et les fonctions s'utilisent de façon similaire (parenthèses obligatoires, arguments, valeur de retour, etc.).

Exemples de méthodes sur les chaînes

```
>>> print(str.islower.__doc__)  
S.islower() -> bool
```

Return True if all cased characters in S are lowercase and there is at least one cased character in S, False otherwise.

```
>>> 'hello'.islower()
```

```
True
```

```
>>> 'Hello'.islower()
```

```
False
```

```
>>> '1'.isdigit()
```

```
True
```

```
>>> 'a'.isdigit()
```

```
False
```

Exemples de méthodes sur les chaînes

Notre fonction `find` devient obsolète grâce à la méthode `find` prédéfinie :

```
>>> find('banane', 'n')  
2  
>>> 'banane'.find('n')  
2
```

Exemples de méthodes sur les chaînes

Cette méthode est plus évoluée que notre fonction `find` :

```
>>> print(str.find.__doc__)  
S.find(sub [,start [,end]]) -> int
```

Return the lowest index in `S` where substring `sub` is found, such that `sub` is contained within `s[start:end]`. Optional arguments `start` and `end` are interpreted as in slice notation.

```
>>> mot = 'banane'  
>>> mot.find('an')  
1  
>>> mot.find('an', 2)  
3
```

Exemples de méthodes sur les chaînes

Exercice : lisez la documentation de la méthode `count` pour écrire une instruction qui permette de compter le nombre de “n” du mot “banane”. Qu’est-ce qui est obligatoire et qu’est-ce qui est optionnel ? Comment le voit-on dans l’en-tête de la méthode ?

```
>>> print(str.count.__doc__)  
S.count(sub[, start[, end]]) -> int
```

Return the number of non-overlapping occurrences of substring sub in string S[start:end]. Optional arguments start and end are interpreted as in slice notation.

Opérateurs sur les chaînes

Opérateur `in`

Les opérateurs `+` et `*` peuvent être appliqués sur des `str` (voir ch. 2). Il existe d'autres opérateurs pour les chaînes.

“`in`” n'est pas utilisé que dans une boucle `for`. C'est aussi un opérateur booléen qui retourne vrai si un élément est inclus dans une séquence. Dans le cas des chaînes, il retourne vrai si une sous-chaîne est incluse dans une chaîne.

```
>>> 'a' in 'banane'
True
>>> 'pepin' in 'banane'
False
```

Opérateur `in`

Par exemple, la fonction suivante affiche toutes les lettres d'un mot qui apparaissent aussi dans un deuxième mot :

```
>>> def in_both(word1, word2):  
    for letter in word1:  
        if letter in word2:  
            print(letter, end = ' ')  
  
>>> in_both('pommes', 'oranges')  
o e s
```

Attention : aux deux façons d'utiliser `in`

- le premier, avec `for`, pour itérer sur chaque lettre
- le deuxième signifie “est-ce une sous-chaîne?”

Comparaison de chaînes

L'opérateur de comparaison `==` fonctionne sur les chaînes :

```
if answer == 'Thank you':  
    print('You are welcome')
```

Les autres opérateurs de comparaison peuvent être également utilisés sur des chaînes : dans ce cas, “plus petit” ou “plus grand” signifie “précède” ou “suit” dans l’ordre alphabétique.

```
>>> 'avoir' < 'etre'  
True  
>>> 'elephant' > 'souris'  
False  
>>> 'avoir' < 'Etre'  
False
```

Remarque : en Python, toutes les lettres majuscules viennent avant les lettres minuscules. La méthode `lower` peut être utile si c’est un problème.

Comparaison de chaînes

Problème

Ecrire une fonction qui permet de trier trois mots, quelle que soit la casse des mots.

Comparaison de chaînes

Problème

Ecrire une fonction qui permet de trier trois mots, quelle que soit la casse des mots.

On commence par écrire une fonction qui permet de trier deux mots et qui gère le problème de la casse.

```
>>> def trie2(mot1, mot2):  
    first = mot1  
    second = mot2  
    if mot1.lower() > mot2.lower():  
        first = mot2  
        second = mot1  
    return (first, second)  
  
>>> trie2('Zebre', 'elephant')  
( 'elephant', 'Zebre' )
```

Comparaison de chaînes

On peut maintenant trier trois mots sans se soucier de la casse : on les trie deux à deux.

```
>>> def trie3(mot1, mot2, mot3):  
    (mot1, mot2) = trie2(mot1, mot2)  
    # le plus grand de mot1 et mot2 est devenu mot2  
  
    (mot2, mot3) = trie2(mot2, mot3)  
    # mot3 est le plus grand des 3 mots  
  
    (mot1, mot2) = trie2(mot1, mot2)  
    # il se peut que (le nouveau) mot2 soit plus petit que mot1  
  
    return (mot1, mot2, mot3)  
  
>>> trie3('a', 'c', 'b')  
('a', 'b', 'c')  
>>> trie3('c', 'b', 'a')  
('a', 'b', 'c')
```

Utiliser des boucles ou la récursivité ?

Utiliser des boucles ou la récursivité ?

Il est toujours possible de transformer un algorithme itératif en un algorithme récursif et vice-versa, même si ce n'est pas toujours facile.

Pour certains problèmes, il est plus simple/naturel d'utiliser des boucles, pour d'autres la récursivité.

En général :

- Utilisez une **boucle** quand vous avez une **séquence d'actions à répéter un nombre de fois défini ou facilement calculable** (par ex., parcourir une chaîne pour travailler sur chaque élément) ;
- Utilisez la récursivité quand le problème **se divise naturellement en sous-problèmes similaires**, et que chaque sous-problème peut être résolu de manière indépendante mais contribue à la solution globale (par ex., calculer Fibonacci ou dessiner une fractale).

Exemple : énumération de combinaisons

Vous disposez d'un ensemble de n éléments distincts. Vous souhaitez énumérer toutes les combinaisons possibles de k éléments parmi ces n éléments.

Problème : Écrivez une fonction récursive en Python qui affiche toutes les combinaisons possibles de k éléments parmi un ensemble de n éléments. Les éléments de l'ensemble sont représentés sous forme de chaîne de caractères.

Exemple : si l'ensemble est $\{A, B, C\}$ et que $k = 2$, les combinaisons possibles sont $\{A, B\}$, $\{A, C\}$, et $\{B, C\}$.

Enumération de combinaisons

```
>>> combinaisons("ABC", 2)
AB
AC
BC
```

A votre avis, qu'est-ce qui est le plus naturel, une (ou des) boucle(s) ou la récursivité ?

Enumération de combinaisons

```
>>> combinaisons("ABC", 2)
AB
AC
BC
```

A votre avis, qu'est-ce qui est le plus naturel, une (ou des) boucle(s) ou la récursivité ?

Il y a clairement un problème qui peut être décomposé en sous-problèmes. La **récursivité** sera plus naturelle ici.

Enumération de combinaisons

Notre fonction récursive va prendre 3 paramètres :

- `elements` : une chaîne avec les éléments parmi lesquels on peut encore choisir pour faire une combinaison (initialement : tous les éléments)
- `k` : le nombre d'éléments qu'il faut encore choisir pour faire une combinaison (initialement, la valeur de `k` au départ)
- `prefix` : une combinaison en construction (paramètre auxiliaire, initialement vide)

```
def combinaisons(elements, k, prefix):  
    ???
```

```
elements = input("Donne moi une chaîne avec des éléments : ")  
k = int(input("Quel k veux-tu ? "))  
combinaisons(elements, k, "")
```

Enumération de combinaisons

Idée : on va construire chaque combinaison selon le principe suivant

- **(cas général)** Pour chaque élément de `elements`, la fonction doit considérer deux options :
 - ➊ Inclure cet élément dans la combinaison actuelle et chercher $k - 1$ éléments parmi les éléments restants.
 - ➋ Ne pas inclure cet élément et chercher k éléments parmi les éléments restants.
- **(base)** Si $k = 0$, cela signifie qu'il n'y a plus d'éléments à sélectionner. La combinaison dans `prefix` peut donc être affichée.
- **(base)** Si le nombre d'éléments restants est inférieur à k , il est impossible de former une combinaison de k éléments, et la fonction doit se terminer sans rien faire.

Enumération de combinaisons

```
def combinaisons(elements, k, prefix = ""):
    if k == 0:
        print(prefix)
        return
    if len(elements) < k:
        return

    # 1. Inclure le premier élément dans la combinaison actuelle
    combinaisons(elements[1:], k - 1, prefix + elements[0])
    # 2. Ne pas inclure le premier élément et passer au suivant
    combinaisons(elements[1:], k, prefix)
```

Comme l'était `x` pour le calcul de la racine carrée (voir chap. 5), le paramètre `prefix` a été ajouté pour construire la solution récursive, même s'il n'est pas décrit dans les entrées de notre problème au départ. C'est un **paramètre auxiliaire** pour lequel on donne une valeur par défaut pour pouvoir appeler `combinaisons("ABC", 2)` par ex.

Erreurs fréquentes et debug

Erreurs d'indices

Quand on utilise les indices pour traverser une séquence, les erreurs d'indices sont fréquentes (indice de fin ou de début).

Voici une fonction qui est supposée comparer deux mots et retourner `True` si l'un des mots est l'inverse de l'autre, mais elle contient deux erreurs :

```
def is_reverse(word1, word2):
    if len(word1) != len(word2):
        return False
    i = 0
    j = len(word2)
    while j > 0:
        if word1[i] != word2[j]:
            return False
        i = i + 1
        j = j - 1
    return True
```

Erreurs d'indices

- le premier `if` vérifie que les 2 mots ont la même longueur, si ce n'est pas le cas, ils ne peuvent être inverses l'un de l'autre
- `i` et `j` sont des indices : `i` traverse `word1` de g. à dr. et `j` traverse `word2` de dr. à g.
- le deuxième `if` vérifie que les lettres “miroir” sont équivalentes, si ce n'est pas le cas, les mots ne peuvent être inverses l'un de l'autre
- si on traverse toute la boucle et que toutes les lettres correspondent, on peut retourner `True`

Erreurs d'indices

Si on teste la fonction avec les mots “pots” et “stop”, on s’attend à obtenir `True`, mais on obtient une erreur d’indice :

```
>>> is_reverse('pots', 'stop')
...
File "<pyshell#211>", line 7, in is_reverse
    if word1[i] != word2[j]:
IndexError: string index out of range
```

Pour déboguer les erreurs d’indices, un réflexe utile est d’afficher les indices.

```
...
    while j > 0:
        print('i:', i, 'j:', j)
...
```

Erreurs d'indices

On observe que lors de la première itération, l'erreur est déjà provoquée. Il s'agit de l'indice de `j` qui est plus grand que `len('stop')-1` :

```
>>> is_reverse('pots', 'stop')
i: 0 j: 4
...
    if word1[i] != word2[j]:
IndexError: string index out of range
```

Correction :

```
def is_reverse(word1, word2):
    ...
    j = len(word2) - 1
    ...
```


Erreurs d'indices

On obtient :

```
>>> is_reverse('pots', 'stop')  
i: 0 j: 3  
i: 1 j: 2  
i: 2 j: 1  
True
```

La réponse est bonne mais la boucle itère 3 fois : est-ce normal ? La fonction est-elle correcte ?

Exercice : utilisez Python Tutor pour corriger complètement cette fonction.

Exercices interactifs

Jouer avec les mots

Le fichier `words.txt`¹ (disponible sur e-learning) contient 113809 mots anglais qui sont considérés comme les mots officiels acceptés dans les mots-croisés.

C'est un fichier "plain text", c-à-d que vous pouvez le lire dans n'importe quel éditeur de texte, mais également via Python.

Nous allons l'utiliser pour résoudre quelques problèmes sur les mots.

1. Fichier disponible publiquement, fourni par Grady Ward pour le projet "Moby"

Lire un fichier de texte, ligne par ligne.

La fonction `open` prend en argument le nom d'un fichier, "ouvre" le fichier (par défaut, en mode lecture : mode `'r'` (read)), et retourne un objet fichier qui permet de le manipuler.

```
>>> fichier = open('words.txt') # cherche ce fichier dans le repertoire courant
>>> print(fichier)
<_io.TextIOWrapper name='words.txt' mode='r' encoding='UTF-8'>
```

Lire un fichier de texte, ligne par ligne.

La méthode `readline` invoquée sur un objet fichier permet de lire une ligne. Lors de la première invocation, la première ligne est retournée. Lors de l'invocation suivante de cette même méthode, la prochaine ligne sera lue.

```
>>> fichier.readline()
'aa\r\n'
>>> fichier.readline()
'aah\r\n'
```

- “aa” est une sorte de lave.
- la séquence `\r\n` représente deux caractères “blancs”² : un “retour chariot” et une nouvelle ligne, qui sépare ce mot du suivant.
- l'objet fichier retient l'endroit où l'on se trouve dans la lecture, la seconde invocation permet d'obtenir le mot suivant.

2. D'autres caractères blancs : un espace ou une tabulation `\t`

Lire un fichier de texte, ligne par ligne.

La méthode `strip` invoquée sur un objet retourne une copie de la chaîne en retirant les caractères “blancs” qui se trouvent au début et en fin de la chaîne.

```
>>> line = fichier.readline()
>>> line.strip()
'aahed'
>>> fichier.readline().strip()
'aahing'
```

- comprenez-vous la dernière instruction ? (*Hint* : que retourne `readline` ?)
- puisque la valeur retournée par une méthode est un objet (tout est objet), la notation point permet d'invoquer des méthodes en cascade à lire de gauche à droite : c'est une forme de la composition.

Lire un fichier de texte, ligne par ligne.

Un fichier est un objet itérable. Une boucle `for` permet de lire les lignes d'un fichier de texte une à une.

```
>>> for line in fichier:  
    print(line.strip())
```

```
aahs  
aal  
aalii  
aaliis  
...  
zymoses  
zymosis  
zymotic  
zymurgies  
zymurgy
```

With open

Depuis Python 3, utiliser `with open...` permet d'ouvrir un fichier mais également de gérer automatiquement certaines choses (comme la fermeture du fichier à la fin du `with` même si une exception se produit).

```
cnt = 0
with open("words.txt") as file:
    for line in file:
        cnt += 1
print(cnt)
```


Exercices

Problème

Ecrivez un script qui lit le fichier `words.txt` et qui n'affiche que les mots qui ont plus de 20 caractères (sans compter les caractères blancs).

Problème

En 1939, Ernest Wright a publié une nouvelle de 50000 mots appelée *Gadsby* qui ne contient pas la lettre “e”. Comme “e” est la lettre la plus commune en anglais (et dans d'autres langues), ce n'était pas une tâche facile ^a.

Ecrivez un script qui n'affiche que les mots qui ne contiennent pas de “e” et calculez le pourcentage de ceux-ci par rapport à l'ensemble des mots du fichier `words.txt`.

a. Georges Perec a fait le même exercice en français, dans son livre *La disparition* (1969) où il définit ce qui a disparu comme “un rond pas tout à fait clos, fini par un trait horizontal”.

Exercices

Solution : les mots de plus de 20 caractères :

```
with open('words.txt') as fichier:
    for line in fichier:
        word = line.strip()
        if len(word) > 20:
            print(word)
```

Exercices

Solution : les mots sans “e” :

```
with open('words.txt') as fichier:
    total = 0
    cnt = 0
    for line in fichier:
        total = total + 1
        word = line.strip()
        if not 'e' in word:
            cnt = cnt + 1

percent = cnt / total * 100.0
print('Pourcentage de mots sans e: %.2f' % percent)
```

Exercices

Problème

Donnez-moi un mot anglais avec 3 doubles lettres consécutives. Je vous donne un couple de mots qui sont presque candidats, mais pas tout à fait. Par exemple, le mot “committee” serait parfait s’il n’y avait pas ce “i” au milieu. Ou “Mississippi” : si on retirait les “i”, cela marcherait.

Il y a cependant au moins un mot qui possède trois paires consécutives de lettres. C’est peut-être le seul mot, ou il peut y en avoir 500. Pourrez-vous me dire, pour le prochain cours, quel est le mot auquel je pense ?

Devinette

Présentation rapide du code du script `mot_devinette.py`.

Ce code n'est pas mis sur Moodle, car vous aller faire une version améliorée vous-même lors des travaux pratiques.

Pour aller plus loin (à lire par soi-même)

Assignment multiple et test d'égalité

Avec les assignments multiples, il faut faire d'autant plus attention à ne pas confondre l'égalité au sens mathématique et l'assignment.

	Relation d'égalité	Assignment
Symétrique	<i>Oui</i> : Si $a = 7$, alors, $7 = a$	<i>Non</i> : $a = 7$ légal, $7 = a$ pas
Booléen	<i>Oui</i> : égalité est soit vraie, soit fausse. En math, si $a = b$, alors a sera <i>toujours</i> égal à b	<i>Non</i> . De plus, peut rendre deux variables égales, mais cela peut changer $a = 5$ $b = a$ # a et b égaux $a = 3$ # a et b ne sont plus égaux

Un peu plus de détails sur `range`

La fonction `range` retourne un objet `range` permettant d'itérer sur une suite d'entiers :

`range(start, stop, step)`

- `start` (optionnel) : premier entier de la suite (par défaut 0)
- `stop` (obligatoire) : dernier entier de la suite **non-compris**
- `step` (optionnel) : entier qui représente l'incrément ou le décrément entre deux valeurs de la suite (par défaut 1)

```
>>> for i in range(1, 5):  
    print(i, end = ' ')
```

```
1 2 3 4
```

```
>>> for i in range(5):  
    print(i, end = ' ')
```

```
0 1 2 3 4
```


Un peu plus de détails sur range

```
>>> for i in range(2, 8, 2):  
    print(i, end = ' ')
```

2 4 6

```
>>> for i in range(8, 0, -1):  
    print(i, end = ' ')
```

8 7 6 5 4 3 2 1

```
>>> for i in range(stop = 9, step = 3):  
    print(i, end = ' ')
```

TypeError: range() does not take keyword arguments

La fonction `range` ne prend pas d'arguments mots-clefs : il faut donc spécifier `start` si on veut définir `step`.

Exemple d'utilisation de `range`

Problème

Ecrire une fonction qui calcule la somme des nombres pairs présents dans l'intervalle $[n, m]$ ($n \leq m$).

```
>>> def sum_even(n, m):  
    if n % 2 == 1:  
        n = n + 1  
    sum = 0  
    for i in range(n, m + 1, 2):  
        sum = sum + i  
    return sum
```

```
>>> sum_even(2, 4)
```

```
6
```

```
>>> sum_even(1, 5)
```

```
6
```

Debug par bisection

Quand la taille d'un programme augmente, il en va de même pour le travail de debug. Une manière de gagner du temps pour trouver un bug, est la méthode dite de "bisection". Par exemple,

- s'il y a 100 lignes de code, et que vous le testez ligne après ligne, cela fera 100 étapes.
- essayez plutôt de couper le problème en deux. Inspectez au milieu du programme une valeur que vous pouvez tester (ajoutez un `print` ou quelque chose qui a un effet vérifiable).
- si le test au milieu est incorrect, le problème devrait se trouver dans la première partie du programme. S'il est correct, il devrait se trouver dans la seconde partie.
- répétez la procédure jusqu'à la localisation précise du problème.
- en pratique, il n'est pas toujours facile (ou possible) de tester le "milieu" du programme. Dans ce cas, choisissez des endroits où ajouter un test est facile.

Données pour les tests unités

- des fichiers comme `words.txt` sont utiles pour créer de nombreux tests unités automatiquement.
- s'assurer de tester tous les “cas spéciaux” : par ex., une méthode qui vérifie si un caractère est présent dans une chaîne devrait tester les cas où : le caractère est présent en début, en fin et au milieu ; tester la chaîne vide ; tester le cas où le caractère n'est pas présent.
- tester aide à trouver des bugs, mais il n'est pas toujours facile de générer de bons ensembles de cas testés, et même si c'est le cas, vous n'avez jamais l'assurance que le programme est correct.

Dijkstra (informaticien célèbre) :

*Program testing can be used to show the presence of bugs,
but never to show their absence !*

Glossaire

Glossaire

incrémenter : mettre à jour une variable en augmentant sa valeur (souvent de 1).

décrémenter : mettre à jour une variable en diminuant sa valeur (souvent de -1).

itération : (ou boucle) exécution répétée d'un ensemble d'instructions.

boucle infinie : une boucle dont la condition de fin n'est jamais satisfaite.

objet : quelque chose qu'une variable peut référer. Pour le moment, un objet est une "valeur" qui possède des attributs et des méthodes.

séquence : ensemble ordonné, c-à-d un ensemble de valeurs où chaque valeur est identifiée par un index entier.

élément : (item) une des valeurs d'une séquence.

tranche : (slice) partie d'une chaîne spécifiée par un intervalle d'indices.

chaîne vide : chaîne sans caractère et de longueur 0, représentée par deux apostrophes.

immuable : propriété d'une séquence dont les éléments ne peuvent être assignés.

méthode : fonction qui est associée à un objet et qui est invoquée en utilisant la notation point.